LIME

Local Interpretable
Model-agnostic Explanations

William Bendinelli

SCC5819 Topics in Artificial Intelligence · ICMC-USP

It passed cross-validation. It failed with users.

Marco Ribeiro, who wrote LIME, tells this on himself. During an internship he trained a model that scored well in cross-validation and fell apart in user testing. After a long debugging, the cause: the model had learned to detect *how* the positive and negative data had been collected — a shortcut that did not exist in the real task.

WHAT HE COULD NOT DO

Tell what his own model was using.

IN HIS OWN WORDS

“Here I was, a PhD student, supposed to be an expert — and it took me the longest time to understand how my model was making predictions.”

Marco Túlio Ribeiro · author of LIME

Two problems, and the second one is worse.

WHAT WE MEASURE

Cross-validation says the model is accurate.

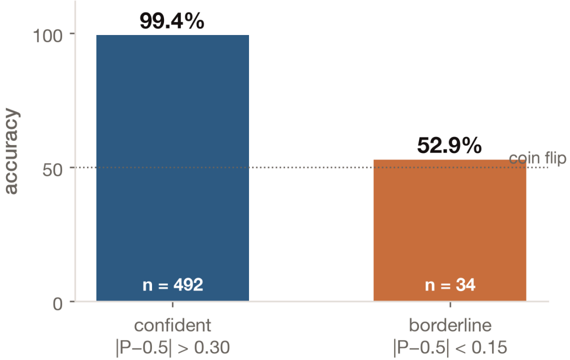
WHAT WE NEED

We need to know it is accurate *for the right reasons*.

ACCURACY ≠ UNDERSTANDING

The same shape of problem, in our data.

Ten-fold cross-validation over all 569 patients gives 95.3% accuracy. Split by how confident each prediction was, that single number comes apart: where the model is sure it is almost never wrong, and where it hesitates it is a coin flip. Those 6% are the patients a clinician would take to a second opinion.



Four words, four commitments.

LIME does not open the model. It builds a simple, readable approximation of how the model behaves in the neighborhood of one prediction — and it does that without needing to know what kind of model it is.

LOCAL	faithful near one point
INTERPRETABLE	readable by a human
MODEL-AGNOSTIC	any model, any data
EXPLANATIONS	input → prediction, legible

01 Do not explain everything

One prediction at a time. Explaining a whole model at once forces you to hide too much.

02 Treat the model as a black box

You give up its internals; you gain the ability to explain any model at all.

03 Perturb, then observe

If you cannot look inside, change the input and watch what the prediction does.

04 Fit something simple

A linear model over those answers — describing the model near this point, not everywhere.

One equation, three demands on the explanation.

$$\xi(x) = \arg\min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g)$$

01 Be faithful to the black box — the loss $\mathcal{L}$ compares g against f

02 Be faithful *here* — the kernel π_x weights nearby samples more

03 Stay simple enough to read — the penalty Ω caps the complexity of g

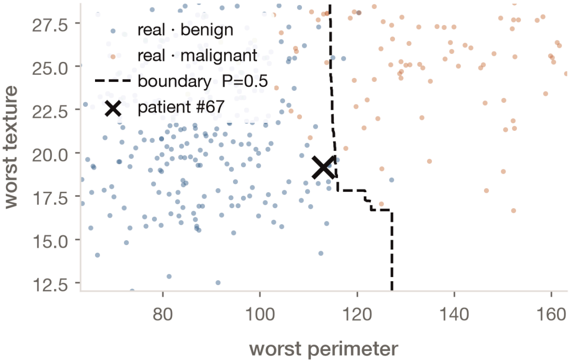
What each term is, concretely.

One configuration choice worth flagging: we pass discretize_continuous=False. The package default bins features into quartiles and phrases explanations as intervals — the form used in Molnar's figures. Keeping them continuous is what lets us draw the explanation as a line.

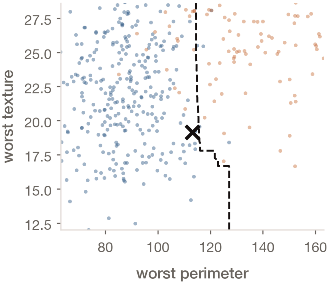
BLACK BOX F	RandomForest, 300 trees
SURROGATE G	$\text{Ridge}, \alpha = 1$
KERNEL π_x	$\sqrt{\exp(-d^2/\nu^2)}$
WIDTH ν	$0.75\sqrt{30} \approx 4.11$
COMPLEXITY Ω	8 of 30 features + L2

One patient, and why she was chosen.

Test patient #67 — predicted benign, truly benign, and sitting on the decision boundary. That is required, not convenient: the fit's P=0.5 contour sits $|g(x)-0.5|/\|w\|$ away from the patient, so a confident case pushes it off the figure entirely. The axes are her rank-1 and rank-3 features; rank 2 correlates with rank 1 at a median of 0.98 across the dataset and would flatten the plot to a ribbon.



THE MODEL



Dashed: the forest's own boundary in this slice.

THE CODE

```
# the only thing we
# ever ask of f
p = model.predict_proba(X)[:, 1]
```

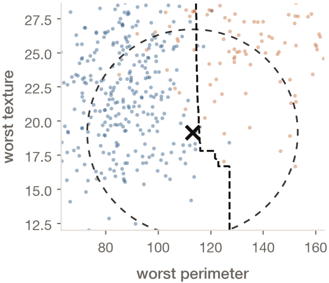
THE IDEA

A staircase, not a curve.

A forest splits one feature at a time, so even 300 averaged trees give elbows. Its exact position moves 0.06σ–0.48σ across seeds.

We never look inside f.

THE MODEL



±1.2σ reference — not LIME's kernel.

THE CODE

```
# one width per feature,
# on its own scale
nu = 0.75 * np.sqrt(30)
```

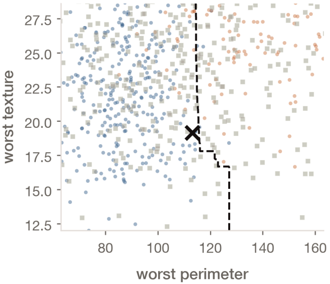
THE IDEA

There is no single radius.

Each feature has its own scale. But note the real kernel is far wider than this frame: the median neighbor sits ~6σ away and still carries a third of the weight.

Remember that width. It returns twice.

THE MODEL



Squares: synthetic neighbors, all 30 features resampled.

THE CODE

```
# centred on the DATASET,
# not on the patient
Z = mu + sigma * rng.normal(
        0, 1, size=(n, 30))
```

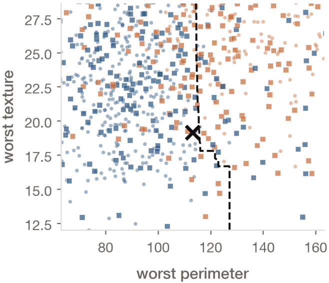
THE IDEA

These are not possible patients.

Sampling each feature independently breaks the correlations that make a tumour measurable: ~76% carry a negative measurement.

This comes back in the limitations.

THE MODEL



Colour: what f predicts for each neighbor.

THE CODE

```
# the one moment the real
# model is queried
fz = model.predict_proba(Z)[:, 1]
```

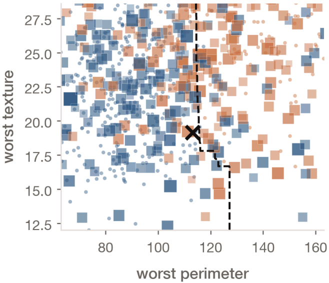
THE IDEA

They do not split along the line.

68% agreement against a 53% baseline. Each square has random values on the other 28 features, while the contour freezes them at hers.

A 2-D slice cannot decide a 30-D prediction.

THE MODEL



Size and opacity encode the weight.

THE CODE

```
d = norm(Zs - x_scaled, axis=1)
w = np.sqrt(
      np.exp(-d**2 / nu**2))
```

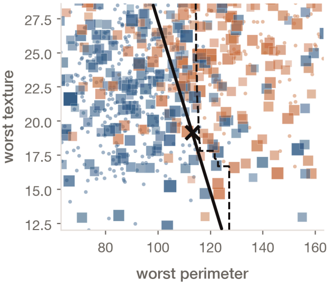
THE IDEA

Closer neighbors count more.

Distance is over all 30 standardized features, not the two on screen — a square that looks far can still be heavy.

Look how little the weights actually vary.

THE MODEL



Solid: the fit, drawn through the patient.

THE CODE

```
Ridge(alpha=1).fit(
    Zs[:, top8], fz,
    sample_weight=w)
```

THE IDEA

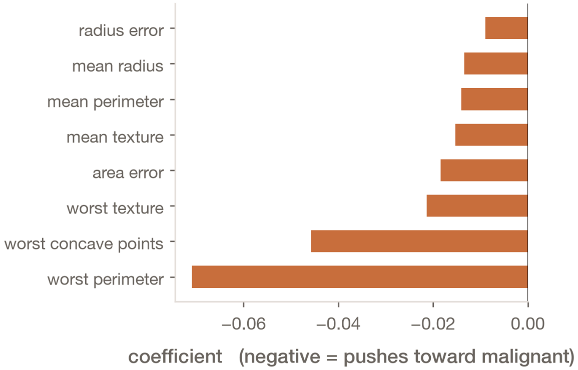
One straight line for a staircase.

Fitted on the 8 selected features, weighted by π. Drawn through the patient rather than at P=0.5 — the next section explains why.

$R^2 = 0.36$

What the library actually hands you.

Not a line — a list of coefficients. Every one of them is negative here: all eight selected measurements push toward malignant, and the model still returns benign. The remaining 22 features and the forest's nonlinearity carry the verdict, and a local linear summary structurally cannot show you that.



A linear fit gives you two things. They are not equally reliable.

DO NOT TRUST

The *level*: g(x) = 0.499 while f(x) = 0.581.

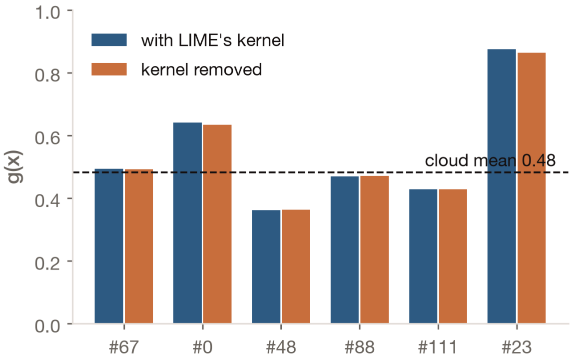
TRUST

The *direction*: 4° from where the model actually changes.

COSINE 0.997 IN-PLANE · 0.57 ACROSS 30-D

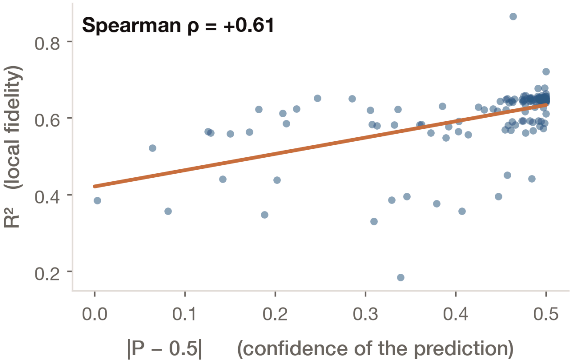
The level is biased, and here is why.

Delete the proximity kernel entirely and almost nothing changes — g(x) moves by ~0.002, the coefficient directions stay aligned at cosine 0.9999. So the fit is effectively global. A global fit explaining a third of the variance shrinks its predictions toward the cloud's mean of 0.48, which is why g(x) lands near 0.5 for almost any patient. The intercept describes the cloud, not her.



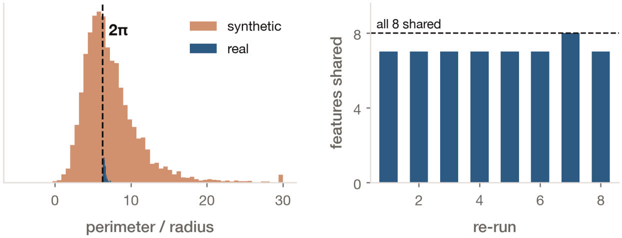
A high R^2 can certify that nothing is happening.

R^2 tracks how confident the prediction was, not how good the explanation is (ρ = +0.61). The intuitive story — the model saturates far from the boundary, so a line fits easily — is false: it never saturates where LIME samples (2 of 5,000 draws, against 71% of real patients), and the weighted variance of f is identical in both groups. The replacement story fails too. The correlation is robust and we cannot say why.



LIME is useful.
It is not neutral.

01 Samples impossible data
02 The kernel barely localizes
03 R^2 measures the wrong thing
04 The explanation wobbles



OFF-MANIFOLD perimeter / radius **INSTABILITY** top-8 across re-runs

Three of the limitations Molnar lists, measured on our own run.

Neighborhood definition, sampling that ignores feature correlation, and instability across runs.

Which number would you report to the board?

Not 95.3% alone. 95.3% together with: on the 6% of patients where the model hesitates it is right about half the time — and here is a tool that tells you which patient you are looking at, along with how much of what it says you should believe.

KEY TAKEAWAY

LIME's value is not that it is never wrong. It is that, once you measure it, you can say precisely how it is wrong.

Thanks!

github.com/wbendinelli/interpretable-ml-lectures

Notebooks, figures and these slides. Every number in this deck is printed by the notebooks in module 03.